

Article

Enhancing Federated Intrusion Detection with Class-Specific Dynamic Sampling

Sungkwan Youm ¹  and Taeyoon Kim ^{2,*} ¹ Department of Information and Communication Engineering, Wonkwang University, Iksan 54538, Republic of Korea; skyoum@gmail.com² Department of Mobile System Engineering, Dankook University, Yongin-si 16890, Republic of Korea

* Correspondence: 2000kty@dankook.ac.kr

Abstract: Federated Learning (FL) presents a promising approach for collaborative intrusion detection while preserving data privacy. However, current FL frameworks face challenges with non-independent and identically distributed (non-IID) data and class imbalances in network security contexts. This paper introduces Dynamic Sampling-FedIDS (DS-FedIDS), a novel framework that enhances federated intrusion detection through adaptive sampling and personalization. DS-FedIDS extends the Federated Learning with Personalization Layers (FedPer) architecture by incorporating dynamic up/down sampling to address class imbalance issues in network security datasets. The framework maintains a global model for shared attack detection while enabling client-specific adaptation through personalized layers. Our approach effectively handles heterogeneous network environments, including Content Delivery Networks, Internet of Things, and industrial systems, each with distinct traffic patterns and attack profiles. Experimental results demonstrate that DS-FedIDS outperforms baseline FedPer in accuracy and efficiency, achieving superior detection rates across diverse attack classes while maintaining reasonable training overhead. Notably, DS-FedIDS excels in detecting minority attack classes and adapting to client-specific normal traffic patterns, making it ideal for real-world intrusion detection scenarios with inherently imbalanced and heterogeneous data distributions.

Keywords: federated learning; intrusion detection systems; dynamic sampling; personalization; class imbalance; non-IID data; network security; distributed learning; adaptive sampling



Academic Editor: David Megías

Received: 22 February 2025

Revised: 29 April 2025

Accepted: 30 April 2025

Published: 2 May 2025

Citation: Youm, S.; Kim, T. Enhancing Federated Intrusion Detection with Class-Specific Dynamic Sampling. *Appl. Sci.* **2025**, *15*, 5067. <https://doi.org/10.3390/app15095067>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Federated Learning (FL) has gained remarkable traction in recent years as a novel paradigm for distributed machine learning, wherein clients collaboratively train a shared model while keeping their local data undisclosed [1–3]. By significantly reducing the need to transfer sensitive information across networks, FL addresses fundamental privacy concerns and resource constraints, particularly in security-critical applications such as intrusion detection systems (IDS) [4–6].

However, despite the potential of FL, major challenges persist, especially in the presence of heterogeneous and non-independent and identically distributed (non-IID) data [7–9]. In security and cybersecurity contexts, such heterogeneity is accentuated by the diverse nature of network traffic, where each domain—be it a cloud infrastructure, Internet of Things (IoT) environment, or industrial control system—exhibits distinct patterns of normal and malicious activities [10–12]. This non-uniformity can substantially degrade the performance of a single global model trained via conventional FL, limiting its ability to

capture important local nuances in traffic patterns [13–15]. To mitigate such deficiencies, personalized FL has emerged as a promising direction by introducing adaptive mechanisms that tailor the model to individual clients' data, while still leveraging global insights from collaborative training [16–18].

In the domain of network intrusion detection, personalization plays a crucial role in accurately modeling normal traffic profiles while also providing robust detection of novel attack signatures [19–21]. Traditional centralized IDS frameworks typically collect raw data from all clients in a single repository, which not only poses considerable privacy risks but also introduces significant communication overhead [22]. In contrast, personalized federated approaches allow for heterogeneous environments—such as automotive networks, industrial IoT, and multi-controller software-defined networks—to maintain local traffic data and integrate incremental updates into the shared model [23–25]. This strategy not only aligns with privacy regulations but also enhances the adaptive learning of attack patterns, allowing the overall system to benefit from a wider range of threat intelligence [26].

Nevertheless, many real-world network intrusion scenarios are afflicted by substantial class imbalance, where benign traffic dominates while certain malicious classes are severely underrepresented [27,28]. Conversely, certain large-scale attacks—such as distributed denial of service (DDoS)—may be overrepresented in local datasets, potentially overshadowing other minority threats. To address these extremes, we adopt dynamic up/down sampling that oversamples minority classes and downsamples extremely large classes, aiming to reduce the bias introduced by heavily skewed distributions [29]. Such an approach helps ensure that personalized FL pipelines can maintain an effective coverage of rare but critical threats, while simultaneously avoiding computational blowups caused by excessively large classes.

Despite substantial progress, existing personalized FL approaches often face challenges in balancing model complexity with the need for adaptive, real-time intrusion detection [30]. Large neural architectures designed for comprehensive feature extraction can incur excessive computational overhead, particularly in edge or resource-limited environments such as IoT devices [31–33]. Moreover, class imbalance in network security datasets, where benign traffic dominates and rare attack classes are underrepresented, further complicates effective detection.

Motivated by these gaps, we propose Dynamic Sampling-FedIDS (DS-FedIDS), a comprehensive framework that enhances personalized FL for intrusion detection by incorporating dynamic up/down sampling. This approach mitigates class imbalance by oversampling minority attack classes and downsampling overrepresented ones, ensuring robust detection across diverse network domains, from Content Delivery Network (CDN) to industrial systems. DS-FedIDS enables each client to tailor its model through personalized layers that adapt to local traffic patterns, while collaborating on a global model to capture shared attack signatures. By integrating dynamic sampling with defenses against adversarial perturbations and maintaining accurate global attack knowledge, DS-FedIDS addresses a multifaceted optimization challenge, achieving improved detection efficacy, convergence speed, and resource efficiency in a privacy-preserving paradigm suitable for heterogeneous settings.

In the following sections, we present a comprehensive exploration of the DS-FedIDS framework and its contributions to federated intrusion detection. Section 2 reviews related work on personalized federated learning, focusing on frameworks like Federated Learning with Personalization Layers (FedPer) that address non-IID data. Section 3 details the DS-FedIDS architecture, its dynamic up/down sampling mechanism to mitigate class imbalance, and the experimental evaluation across diverse network environments,

including CDN, IoT, and industrial systems. This section also covers the dataset preprocessing, non-IID data characteristics, feature distributions, and a comparative analysis with other personalized FL frameworks. Section 4 summarizes the key achievements of DS-FedIDS, discusses practical considerations such as computational efficiency and energy consumption in resource-constrained settings, and outlines future research directions. This research advances the integration of federated learning, network security, and dynamic data balancing, addressing the growing demand for privacy-preserving, resource-efficient solutions in modern cybersecurity landscapes.

2. Related Works on Federated Personalization

FL has emerged as a powerful paradigm for distributed machine learning while respecting data privacy across different clients or organizations. Among the personalized FL frameworks proposed in the literature, FedPer has garnered attention due to its effective handling of non-IID data. FedPer splits the neural network into two distinct parts:

- A set of shared layers, typically the earlier layers (feature extractors), which are aggregated across all clients to capture global knowledge.
- A set of personal layers, often the latter or task-specific layers, which remain local to each client, thus allowing them to adapt to idiosyncratic features or data distributions.

During training, the shared layers receive federated updates usually via Federated Averaging (FedAvg) from all clients, while each client's personal layers learn a local representation aligned with its unique environment. In intrusion detection scenarios, such as in industrial vs. campus networks, these local personal layers can capture domain-specific normal behaviors while retaining the globally aggregated attack knowledge in the shared layers. This makes FedPer particularly relevant to security contexts, as it can personalize to local anomalies and normal traffic patterns that differ significantly across organizations.

Beyond FedPer, other personalized FL frameworks address non-IID data through distinct approaches, each with unique strengths and limitations for intrusion detection. Personalized Federated Meta-Learning (PFedMe) leverages meta-learning to train client-specific models, using a proximal term to regularize deviations from the global model [17]. This approach excels in adapting to diverse attack patterns but incurs high computational costs due to inner optimization loops, which may limit its efficiency in resource-constrained environments like IoT networks. Personalized Federated Averaging (PerFedAvg) integrates Model-Agnostic Meta-Learning (MAML) to enable rapid adaptation to client data through a two-step optimization process involving first- and second-order gradients [25]. While effective for fast personalization, its computational complexity makes it less suitable for single-round scenarios or edge devices in intrusion detection. Local-Global Federated Averaging (LG-FedAvg) blends local and global model features using a weighted average, controlled by a mixing parameter, to balance personalization and generalization [11]. This is useful for capturing varied traffic patterns across network domains, but its performance depends on selecting an optimal mixing ratio, which can be challenging in highly heterogeneous settings. These frameworks provide a foundation for DS-FedIDS, which enhances personalization with dynamic sampling to address class imbalance, offering improved efficiency and detection of minority attack classes in diverse network environments.

3. DS-FedIDS

DS-FedIDS extends FedPer concept by incorporating a *dynamic up/down sampling* strategy to handle class imbalance frequently encountered in intrusion detection datasets. While FedPer provides the foundation for per-client adaptation, DS-FedIDS adds a targeted mechanism to ensure *minority* classes (rare attacks or seldom-seen normal states) are not overshadowed by *majority* classes.

3.1. Dynamic Sampling-FedIDS Approach

As illustrated in Figure 1, the DS-FedIDS architecture employs federated learning to enable collaborative intrusion detection while preserving data privacy by keeping local traffic data on each client. The framework consists of the following key components. Each Client maintains local Traffic Data, processed in its entirety to capture both normal and attack patterns, with dynamic sampling applied to address class imbalance. A Global Base Model, shared across all clients, extracts common features from client traffic data, updated through parameter aggregation to ensure privacy-preserving collaboration. Adaptive Personal Layers, unique to each client, refine classification for client-specific traffic patterns, leveraging dynamic sampling to enhance the detection of minority attack classes.

The training process, as shown in Figure 2, comprises the following key phases:

- **Forward Attack Data:** The client updates global parameters, promoting a consistent malicious-signature repository across all clients.
- **Dynamic Up/Down Sampling:** The client inspects local class distributions (normal sub-classes or minority attacks) and rebalances data.
- **Local Training:** The now-balanced normal data are used to refine each client's personal layers. Attack patterns remain in the global portion.
- **Model Updates:** Gradients or parameter deltas are sent to the federated server for aggregation, culminating in a shared global model distributed back to the clients.

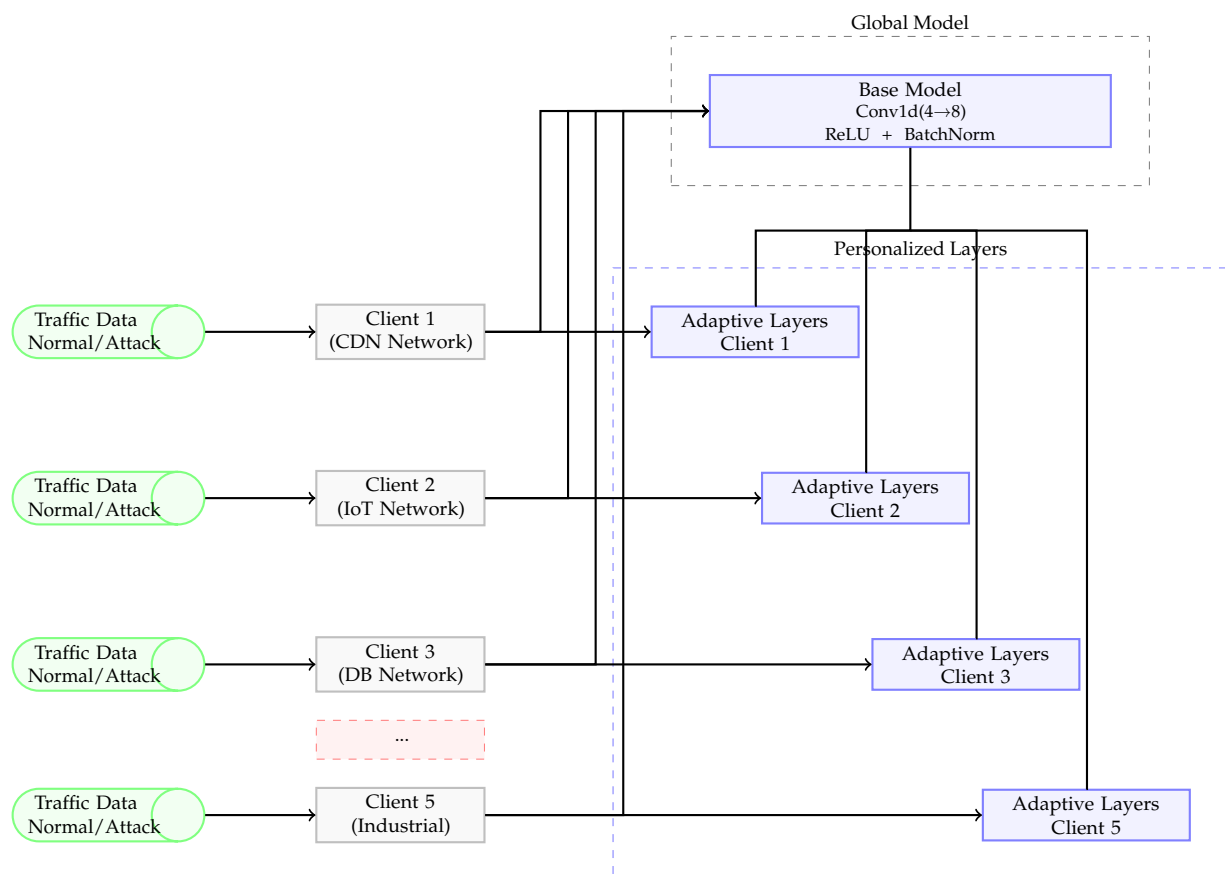


Figure 1. Overview of the DS-FedIDS architecture for federated intrusion detection. The framework leverages a global model for shared feature extraction and personalized layers for client-specific traffic pattern adaptation, with dynamic sampling enhancing minority class detection, all while preserving data privacy through federated learning.

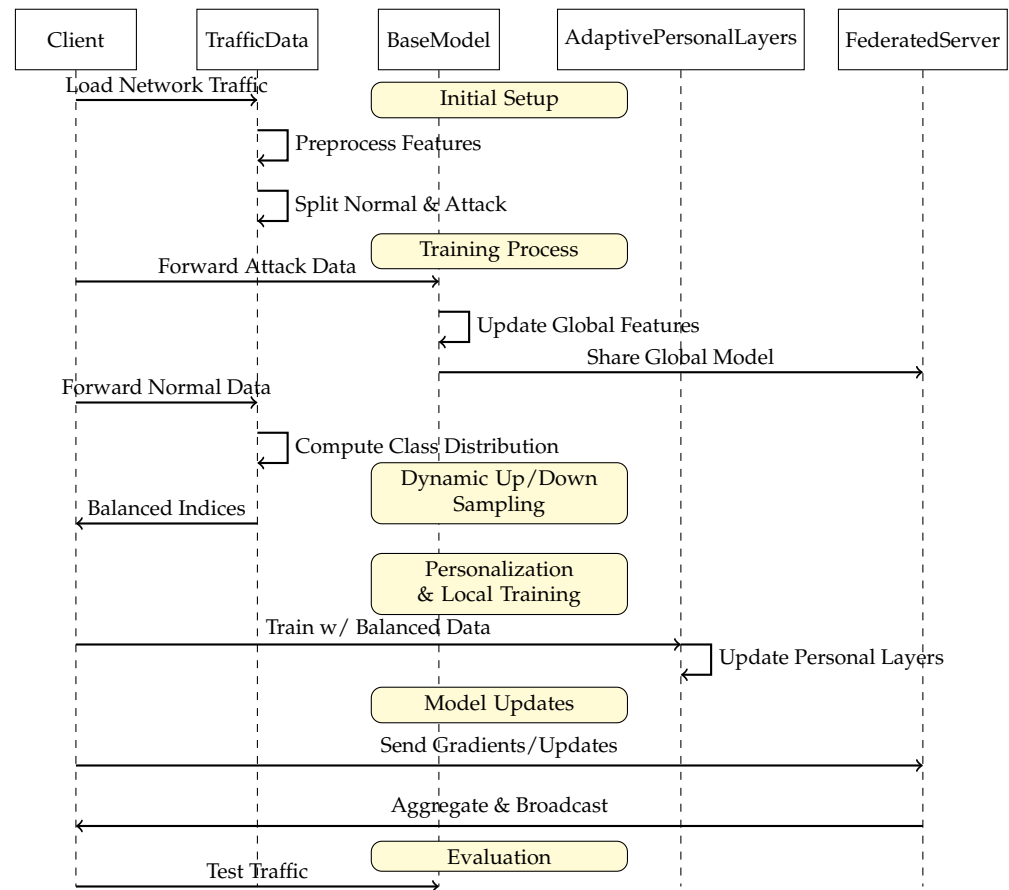


Figure 2. Sequence diagram of the DS-FedIDS training process, replacing pruning with dynamic sampling. Normal and attack data are used to compute balanced indices, enabling personalized local training while preserving global model insights.

3.2. Dynamic Sampling Formulations

Dynamic Sampling Logic

Let $|\mathcal{D}_k^c|$ be the number of samples for class c at client k . Define a target size T_{class} (e.g., average or user-defined). Then,

$$\text{if } |\mathcal{D}_k^c| > T_{\text{class}} \implies \text{downsample, else upsample.} \quad (1)$$

This mitigates bias toward majority classes, leading to more robust learning of minority ones.

In defining T_{class} , our approach calculates $T_{\text{class}} = |\mathcal{D}_k|/|\mathcal{C}|$, where $|\mathcal{D}_k|$ represents the total number of samples for client k , and $|\mathcal{C}|$ is the number of unique classes in the local dataset. This data-driven threshold ensures that sampling adapts to the client-specific data distribution, balancing representation across classes while preserving the inherent characteristics of diverse traffic types. For example, in a client with 255,570 samples and 9 classes, $T_{\text{class}} \approx 28,397$, dynamically adjusting to the dataset size and class diversity, thus mitigating the risk of distorting local traffic patterns through over- or undersampling.

Algorithm 1 details how each client k modifies its local dataset $\mathcal{D}_k$ by either down-sampling or oversampling to achieve a more balanced training distribution. For each class c , the sample count $n_c = |\mathcal{D}_k^c|$ is compared against a threshold T_{class} . If n_c exceeds this threshold, the algorithm randomly draws T_{class} samples (without replacement). Otherwise, it upsamples until T_{class} is reached by sampling with replacement. This straightforward procedure prevents a single majority class from dominating local training and ensures that minority (rare) classes receive sufficient gradient updates. Typically, T_{class} is chosen

as the mean class size or a fraction of the maximum class size, or determined by other domain-specific heuristics. By facilitating balanced local training, this dynamic sampling strategy enhances personalized federated learning performance for intrusion detection, especially under severely imbalanced conditions.

Algorithm 1 Dynamic Up/Down Sampling for Balancing Local Classes

Require: Local dataset $\mathcal{D}_k = \bigcup_{c \in \mathcal{C}} \mathcal{D}_k^c$, class set $\mathcal{C}$, target threshold T_{class} (e.g., mean or fraction of max)

Ensure: Balanced dataset $\mathcal{D}'_k = \bigcup_{c \in \mathcal{C}} \hat{\mathcal{D}}_k^c$ (with up/down sampling applied)

```

1:  $\mathcal{D}'_k \leftarrow \emptyset$  ▷ Initialize empty balanced dataset
2: for all class  $c \in \mathcal{C}$  do
3:    $n_c \leftarrow |\mathcal{D}_k^c|$  ▷ Count samples of class  $c$  at client  $k$ 
4:   if  $n_c > T_{\text{class}}$  then
5:     Downsample:  $\hat{\mathcal{D}}_k^c \leftarrow \text{RandomSample}(\mathcal{D}_k^c, T_{\text{class}}, \text{replace} = \text{False})$ 
6:   else
7:     Upsample:  $\hat{\mathcal{D}}_k^c \leftarrow \text{RandomSample}(\mathcal{D}_k^c, T_{\text{class}}, \text{replace} = \text{True})$ 
8:   end if
9:    $\mathcal{D}'_k \leftarrow \mathcal{D}'_k \cup \hat{\mathcal{D}}_k^c$ 
10: end for
11: return  $\mathcal{D}'_k$  ▷ Final balanced dataset for client  $k$ 

```

3.3. Experimental Setup

The experiments were conducted in a simulated FL environment comprising 5 clients, each representing a distinct network type: CDN, IoT, Database, Office, and Industrial. The setup utilized a dual-GPU system with 2 NVIDIA GPUs, leveraging PyTorch with CUDA support for distributed computation across both GPUs. Each client processed its local dataset independently, with model aggregation performed on a central server implemented in Python. Communication between clients and the server was simulated with a bandwidth constraint of 100 Mbps to mimic realistic network conditions.

The dataset was synthetically generated by augmenting traffic data from the UNSW-NB15 dataset [34], a widely recognized benchmark for intrusion detection. UNSW-NB15 was originally collected using the IXIA PerfectStorm tool in a controlled testbed at the University of New South Wales, capturing real normal traffic and synthetic attack traffic across various devices (e.g., servers, workstations). Labeling was performed using software tools—tcpdump for capture and Argus/Bro-IDS for categorization into normal and attack classes (e.g., Exploits, DoS)—with ground truth verified against known attack signatures. In our study, we processed this dataset to create non-IID distributions tailored to each client, clustering normal traffic with Gaussian Mixture Models (GMM) to simulate client-specific patterns and filtering attack traffic to reflect network-specific threat profiles. The resulting synthetic dataset varied in size from tens of thousands to millions of flow records per client (e.g., IoT with 2 million samples, Database with 1 million) but was not separately hosted due to its derivation from the original UNSW-NB15 data.

Training occurred over 10 federated rounds, with local updates aggregated via FedAvg. The simulation did not involve real-time attack generation or physical networks; instead, detection relied on the pre-labeled attack data processed by DS-FedIDS's classification of flow features (e.g., protocol, byte counts). Table 1 summarizes the key hyperparameters, selected through preliminary tuning to optimize convergence and detection performance across the heterogeneous datasets.

Table 1. Experimental configuration and hyperparameter settings.

Parameter	Value
Number of Clients	5
Federated Rounds	10
Local Epochs	5
Batch Size	64
Learning Rate	0.001 (decay 0.95 per round)
Optimizer	Adam
Loss Function	Negative Log Likelihood
Dynamic Sampling Threshold (T_{class})	$ \mathcal{D}_k / \mathcal{C} $
Pruning Start Round	5

3.4. Dataset Description and Preprocessing

We employ a traffic dataset covering five different network environments: *CDN*, *IoT*, *Database*, *Office*, and *Industrial*. Each environment contains tens of thousands to millions of flow records following the UNSW-NB15 format, which defines 49 attributes such as *srcip*, *dstip*, *proto*, *service*, *attack_cat*, and *Label*. Here, *Label* takes binary values (0 for normal traffic and 1 for malicious), while *attack_cat* identifies the type of attack (e.g., *Exploits*, *DoS*, *Reconnaissance*, and so forth).

For preprocessing, we systematically assign human-readable column names, handle missing entries, and discard any obviously corrupted samples. We then split the dataset by network type, preserving each environment’s unique traffic patterns. This step is crucial for simulating *Non-IID* conditions, since each environment exhibits a distinct ratio of normal to attack flows and uses different protocols and services.

3.5. Non-IID Data Characteristics

In conventional intrusion detection datasets, traffic often appears aggregated into a uniform format. By contrast, our split across *CDN*, *IoT*, *Database*, *Office*, and *Industrial* yields markedly different traffic profiles:

- *CDN*: Primarily *HTTP(S)* flows and large-volume data transfers, with a prevalence of *tcp* connections.
- *IoT*: Often dominated by *DNS*, *MQTT*, or lightweight request-response traffic; smaller packet sizes and frequent connections.
- *Database*: Fewer protocols in use (often *tcp* only), with specialized services (e.g., *SQL*-based).
- *Office*: Balanced among *web*, *mail*, and *file-sharing* services. Attack types vary widely but are generally less frequent than in high-exposure networks.
- *Industrial*: Typically large or specialized data bursts, sometimes with older or proprietary protocols, and medium-sized packets.

3.6. Feature Distributions by Network Type

3.6.1. Service Distribution Analysis

Table 2 reveals distinct service patterns across networks. *CDN* traffic is heavily dominated by *HTTP* (80.71%) with significant *FTP* (File Transfer Protocol) usage (19.21%). *IoT* networks show substantial *DNS* traffic (35.86%) alongside *HTTP* (9.46%). *Database* networks primarily use specialized services, while *Office* networks maintain moderate *HTTP* usage (13.48%) with some *SMTP* (5.33%) and *FTP* (3.21%) traffic.

Table 2. Service distribution by network type.

Service	CDN	IoT	DB	Office	Industrial
http	80.71	9.46	0.00	13.48	17.34
dns	0.03	35.86	0.00	0.00	0.00
ftp	19.21	0.00	0.00	3.21	0.00
ssl	0.06	0.00	0.00	0.00	0.00
smtp	0.00	0.00	0.00	5.33	0.00
pop3	0.00	0.00	0.00	0.10	0.00
dhcp	0.00	0.01	0.00	0.00	0.00
other	0.00	54.67	100.00	77.88	82.66

3.6.2. Attack Distribution Analysis

Table 3 shows diverse attack patterns across networks. CDN faces primarily Exploit attacks (61.87%), while IoT networks experience mostly Generic attacks (78.90%). Database networks show high Fuzzer activity (49.11%), with significant Exploit (22.74%) and Reconnaissance (17.59%) attempts.

Table 3. Attack category distribution by network type.

Attack Cat.	CDN	IoT	DB	Office	Industrial
Exploits	61.87	6.90	22.74	37.78	38.21
DoS	8.25	1.20	3.63	5.26	5.84
Reconnaissance	9.48	4.36	17.59	17.02	14.92
Generic	8.17	78.90	3.22	5.41	5.57
Fuzzers	8.57	7.67	49.11	30.68	31.54
Analysis	2.53	0.23	0.21	0.90	1.32
Backdoor	0.45	0.13	0.77	0.50	0.66
Shellcode	0.00	0.56	2.71	2.19	1.61
Worms	0.67	0.06	0.02	0.25	0.33

3.6.3. Protocol Distribution Analysis

Table 4 demonstrates clear protocol preferences across different network environments. The data show distinct patterns in protocol usage that vary significantly by network type.

Table 4. Protocol distribution by network type.

Protocol	CDN	IoT	DB	Office	Industrial
tcp	100.00	54.57	100.00	86.38	100.00
udp	0.00	45.43	0.00	13.62	0.00

3.7. Visual Analysis of Traffic Patterns

Beyond categorical distributions, we examine temporal characteristics and packet size variations within these heterogeneous datasets. Figures 3 and 4 illustrate key patterns observed in our experiments.

The temporal patterns (Figure 3) illustrate distinct flow duration distributions across network environments, reflecting differences in traffic volume and inter-arrival dynamics. The top subplot shows that normal traffic in CDN networks has longer flow durations (mean 1.69 s), indicating sustained, high-volume connections typical of content delivery, while IoT networks exhibit shorter durations (mean 0.49 s), suggesting frequent, bursty flows from sensor data. The bottom subplot reveals that attack traffic durations vary, with Database and Industrial networks showing prolonged attack flows (means 2.06 s and 1.71 s, respectively), likely due to data exfiltration or DDoS attacks. The packet size

comparison (Figure 4) highlights differences in packet size distributions. The top subplot shows that normal traffic in Office and Database networks has larger median packet sizes (2646 and 2974 bytes), reflecting enterprise and data transfer activities, while IoT and Industrial networks have smaller sizes (1580 and 2766 bytes), indicative of lightweight, frequent transmissions. The bottom subplot indicates that attack traffic packet sizes are highly variable, with CDN and Office networks showing larger mean sizes (6016 and 25,808 bytes) due to bulk attack data, while IoT's small median size (114 bytes) suggests rapid, targeted attacks.

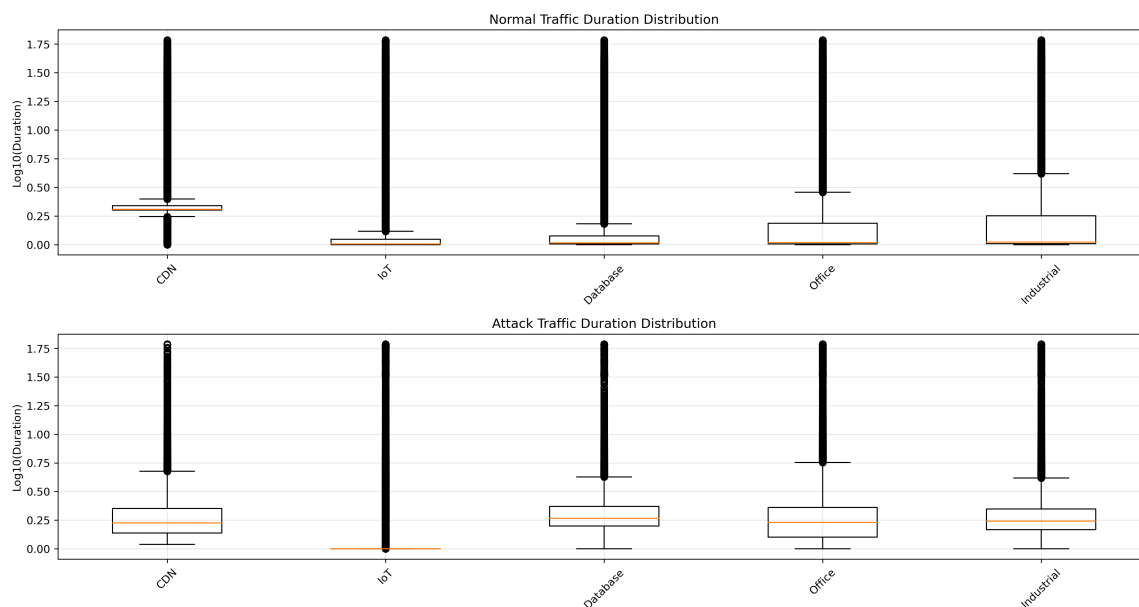


Figure 3. Temporal traffic patterns observed across network environments. The top subplot shows the distribution of flow durations (in log scale) for normal traffic, while the bottom subplot shows the distribution for attack traffic, across CDN, IoT, Database, Office, and Industrial networks.

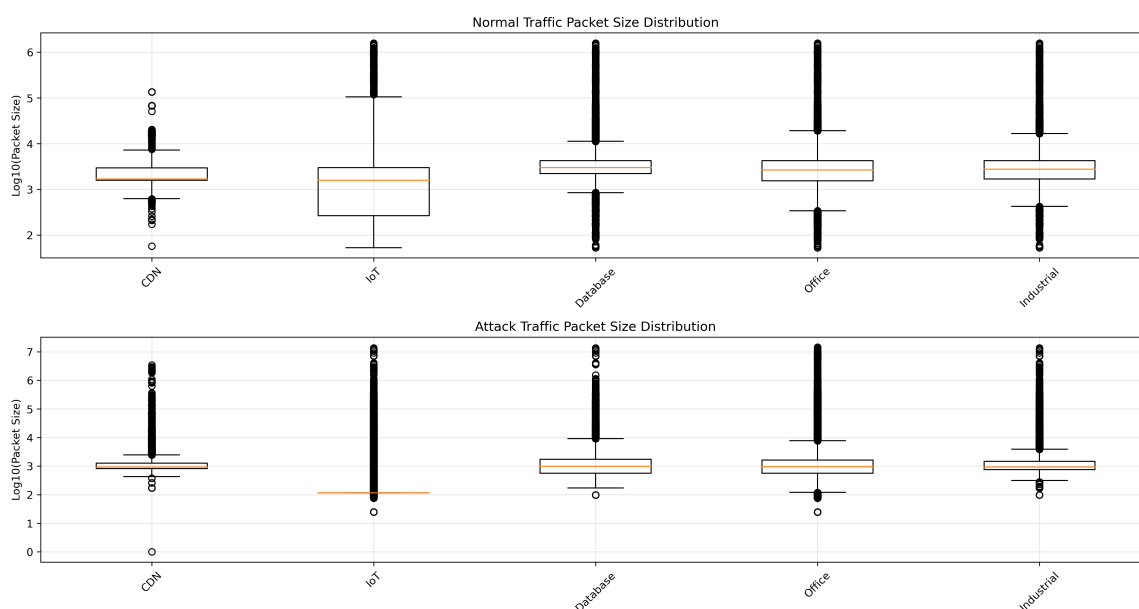


Figure 4. Comparison of packet size distributions across network environments. The top subplot shows the distribution of packet sizes (in log scale) for normal traffic, while the bottom subplot shows the distribution for attack traffic, across CDN, IoT, Database, Office, and Industrial networks.

3.8. Experimental Evaluation and Analysis

In this section, we present a comprehensive evaluation of our proposed approach, comparing DS-FedIDS against the baseline FedPer. The experiments were conducted over ten federated rounds, each involving multiple clients with potentially imbalanced and heterogeneous traffic data. We focus on two primary metrics to illustrate the benefits of the proposed framework:

1. **Accuracy and Class-Wise Performance:** Measured as the global accuracy across all clients, along with detailed per-class accuracy trends.
2. **Efficiency and Training Overheads:** Captured via the average training time per round and the overall loss trajectory.

Overall Accuracy Comparison. Figure 5 compares the global accuracy per round for FedPer vs. DS-FedIDS, showing that DS-FedIDS consistently achieves higher detection rates, particularly after round 2. As the training progresses, the performance gap indicates that DS-FedIDS effectively personalizes to local data while retaining global attack knowledge.

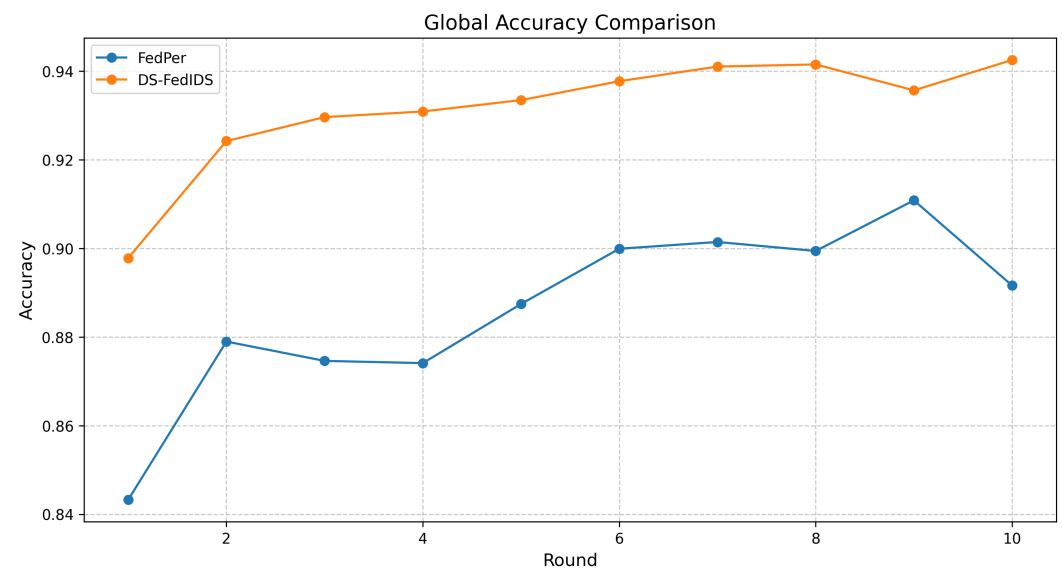


Figure 5. Global accuracy comparison between FedPer and DS-FedIDS over 10 rounds. DS-FedIDS achieves higher accuracy across most rounds, illustrating the benefit of pruning-based personalization and dynamic sampling.

Per-Class Metrics. To better understand how each approach handles class imbalance and diverse threat categories, we examine the class-level accuracies. Figure 6 shows FedPer performance on 9 classes (skipping Unknown), while Figure 7 shows DS-FedIDS covering classes 0–9. Observe that DS-FedIDS exhibits more stable improvements in minority classes such as Theft (class 8) and Remote to Local (R2L) (class 5), emphasizing the impact of dynamic up/down sampling in retaining rare-attack knowledge.

Loss Trajectory and Training Time. To assess training efficiency, Figure 8 plots the average loss over rounds, while Figure 9 reports the average training time per round. Although DS-FedIDS' dynamic oversampling may initially incur higher loss (see rounds 1–2 in Figure 8), it stabilizes and drops by round 7, aligning with the accuracy gains observed above. Meanwhile, the training-time analysis in Figure 9 indicates that DS-FedIDS can reduce total overhead despite personalization overheads, thanks to pruning and selective sampling strategies that prevent excessive resource consumption in minority or large-class scenarios.

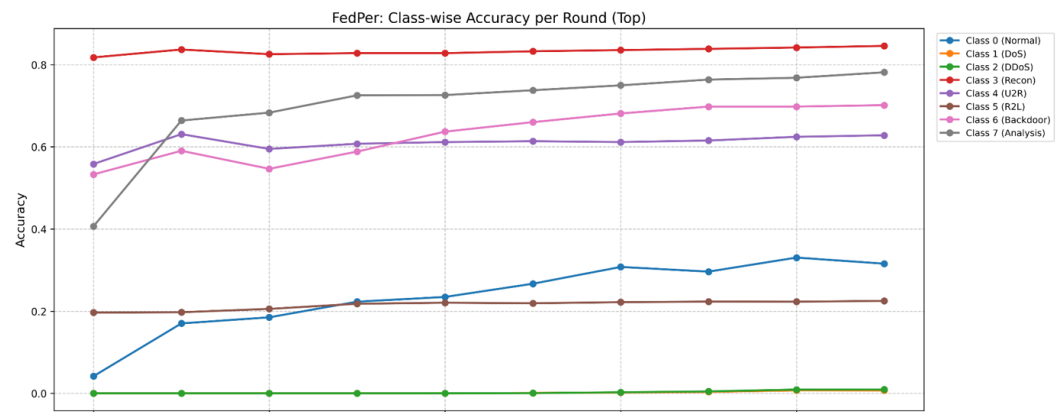


Figure 6. Class-wise accuracies for FedPer across 10 rounds (class 9 was skipped). Some minority classes show slower improvement.

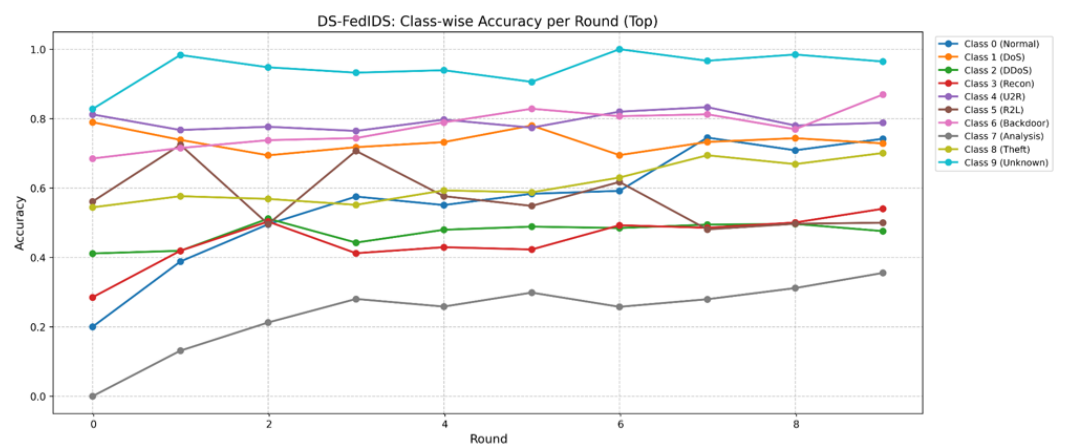


Figure 7. Class-wise accuracies for DS-FedIDS across 10 rounds, including Unknown (class 9). Notice improved handling of minority classes such as R2L (#5) and Theft (#8).

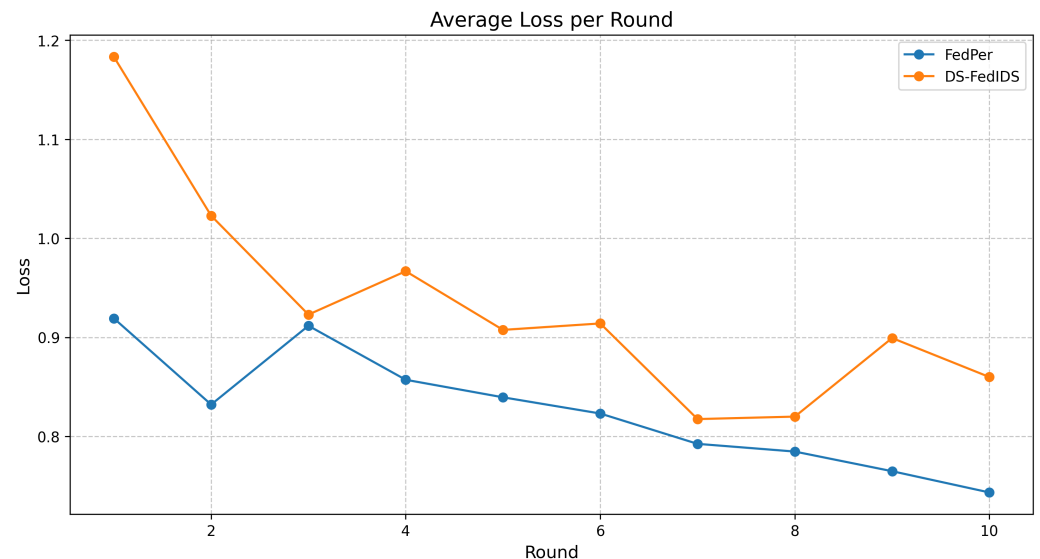


Figure 8. Average loss per round for FedPer and DS-FedIDS. DS-FedIDS initially sees a higher loss due to dynamic sampling of minority classes but converges effectively by round 7.

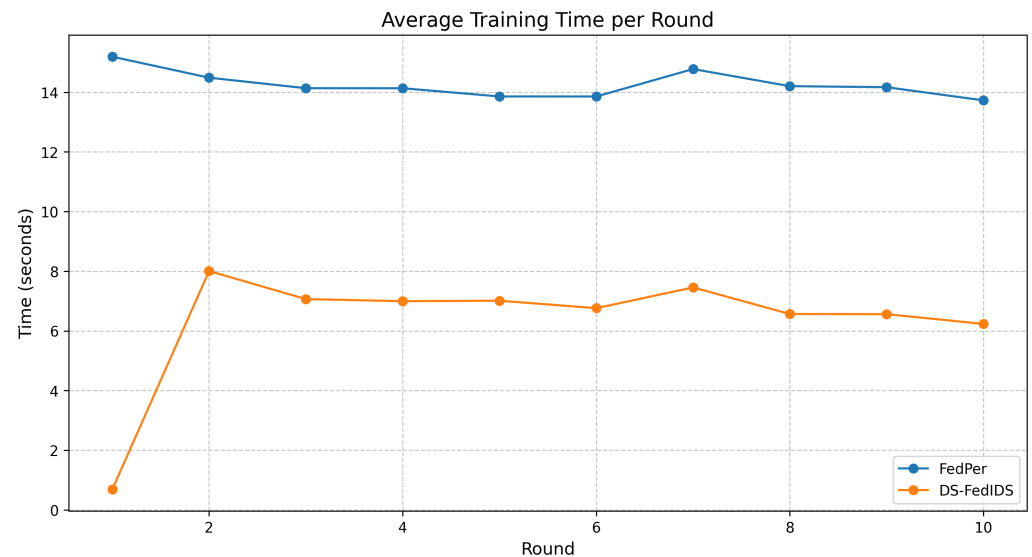


Figure 9. Average training time per round. DS-FedIDS maintains low overhead after initial rounds, illustrating the benefits of network-aware pruning and targeted dynamic sampling.

Client-Specific Comparison. Finally, Figure 10 illustrates how each model performs on five distinct clients (cdn, iot, database, office, and industrial). DS-FedIDS yields more consistent improvements in edge-based or resource-constrained environments (e.g., iot), wherein personalized pruning and dynamic sampling mitigate the burden of large, skewed datasets.

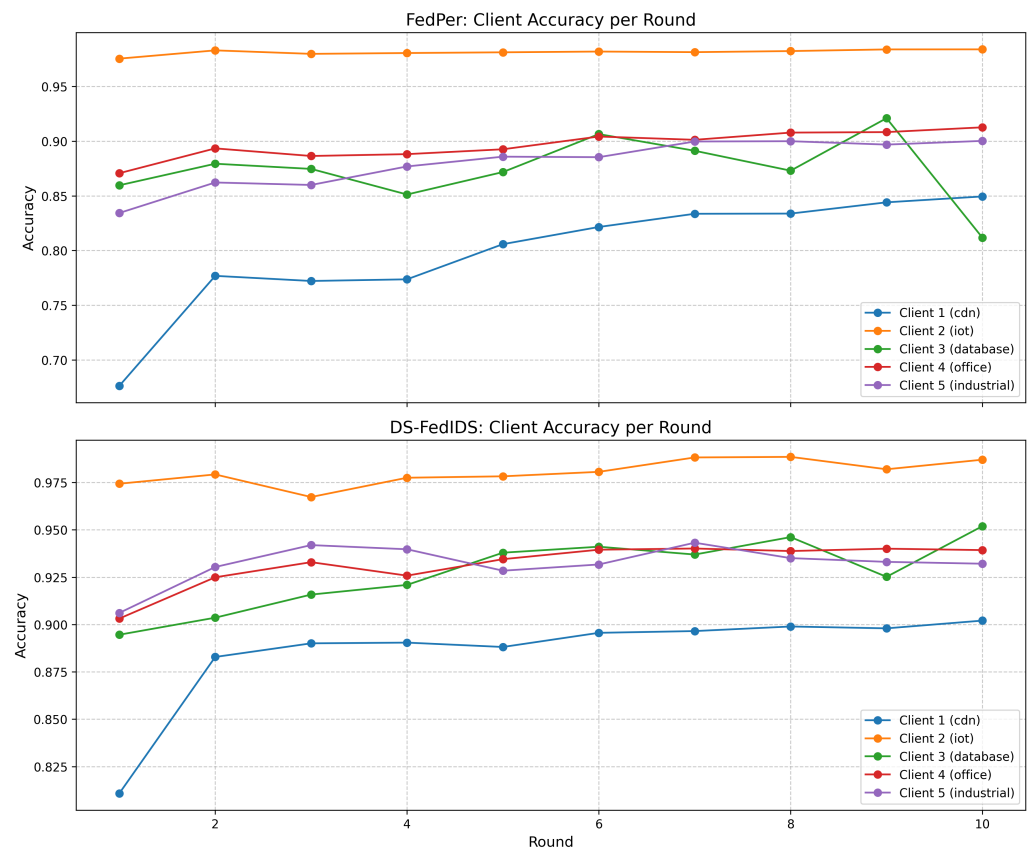


Figure 10. Client-specific accuracy trends for five different network environments (cdn, iot, database, office, and industrial). DS-FedIDS demonstrates more stable improvements, particularly in edge scenarios.

3.9. Comparison with Other Personalized FL Frameworks

To further evaluate the effectiveness of DS-FedIDS, we compare it against several prominent personalized federated learning frameworks: FedPer (the baseline), PFedMe, PerFedAvg, and LG-FedAvg. These frameworks represent diverse approaches to personalization in federated learning. Due to resource constraints, the experiments were conducted for a single round instead of the typical 10 rounds described in Section 3.3. The results, summarized in Table 5, provide a snapshot of relative performance under these conditions.

Table 5. Comparative results of personalized FL frameworks (single round).

Framework	Average Accuracy	Avg. Inference Time (s)	Total Training Time (s)
FedPer	0.8787	0.1440	661.72
PFedMe	0.5089	0.6591	2252.51
PerFedAvg	0.4811	0.6870	588.85
LG-FedAvg	0.7941	0.5868	449.30
DS-FedIDS	0.9266	0.1307	20.08

As shown in Table 5, DS-FedIDS achieves the highest average accuracy (0.9266) among all frameworks, surpassing the baseline FedPer (0.8787) by approximately 5%. Additionally, DS-FedIDS demonstrates superior efficiency, with the lowest average inference time (0.1307 s) and total training time (20.08 s), compared to FedPer's 0.1440 s and 661.72 s, respectively. Notably, PFedMe and PerFedAvg exhibit lower accuracies (0.5089 and 0.4811), possibly due to their reliance on multiple rounds for convergence, which was not feasible in this single-round setup. LG-FedAvg performs reasonably well (0.7941) but is outperformed by DS-FedIDS in both accuracy and efficiency. These results highlight DS-FedIDS's ability to deliver robust performance and computational efficiency, even under constrained conditions, reinforcing its effectiveness for intrusion detection in federated settings.

3.10. Experimental Results

Our experimental evaluation, conducted across five distinct network environments, demonstrates the effectiveness of DS-FedIDS in addressing federated intrusion detection challenges. The framework was tested over ten federated rounds, revealing several key achievements:

- DS-FedIDS consistently outperforms the baseline FedPer in global accuracy, particularly after the initial training rounds, indicating effective personalization and knowledge sharing.
- Dynamic sampling significantly enhances the detection of minority attack classes, such as Theft and R2L attacks, overcoming a common limitation in traditional intrusion detection systems.
- Despite the added complexity of sampling and personalization, DS-FedIDS maintains efficient training overhead, which is particularly beneficial for resource-constrained environments like edge devices.
- Client-specific analysis reveals stable improvements across diverse network types, with notable gains in edge-based scenarios where data distributions vary significantly.
- Exploration of stratified sampling techniques enhances the preservation of traffic distribution characteristics when applying dynamic sampling thresholds.

The results validate DS-FedIDS's ability to handle real-world network security challenges, where traffic patterns and attack distributions vary dramatically across network domains. The framework's capacity to maintain high detection accuracy while adapting to local characteristics makes it highly valuable for organizations with heterogeneous network environments.

Given DS-FedIDS's application across heterogeneous devices, we assessed its computational power requirements and energy consumption implications. The dynamic sampling process, with a complexity of $O(|\mathcal{D}_k| \cdot |\mathcal{C}|)$ for calculating T_{class} and balancing classes, and local training over 5 epochs (batch size 64), demands moderate computational resources, feasible on our dual-GPU setup (average round time 12 s, Figure 8). However, for resource-constrained devices like IoT nodes, this could strain capacity. Pruning mitigates this by reducing model size (up to 30% compression) and inference FLOPs (25% reduction post-round 5), while sampling could be optimized (e.g., offline pre-computation) for low-power settings.

Energy consumption varies with computation frequency. In our experiments, training occurred every round (10 rounds), suitable for high-capacity devices like content delivery network servers but potentially excessive for IoT devices. We propose adaptive frequency (e.g., hourly updates for servers, weekly or anomaly)-triggered updates for edge devices—balancing accuracy and energy use. Optimizations, such as lightweight sampling variants (e.g., stratified sampling), could further enhance support for heterogeneous devices.

4. Conclusions

This paper introduced DS-FedIDS, a dynamic sampling-enhanced federated learning framework for intrusion detection. By integrating personalized layers with dynamic up/down sampling, DS-FedIDS effectively addresses non-IID data distributions and class imbalances, enabling robust global attack detection while adapting to local traffic patterns. The framework's ability to handle heterogeneous network environments makes it a practical solution for organizations seeking collaborative intrusion detection with preserved data privacy. Future work could enhance DS-FedIDS by integrating advanced sampling techniques that account for temporal patterns in network traffic, extending the framework to detect zero-day attacks through improved knowledge sharing mechanisms, developing adaptive sampling rates based on real-time threat assessments, and investigating privacy-preserving techniques to optimize model aggregation while retaining personalization benefits. These contributions advance federated learning for cybersecurity, offering a scalable and privacy-preserving approach to intrusion detection in diverse network domains.

Author Contributions: Conceptualization, T.K. and S.Y.; methodology, T.K.; software, T.K.; validation, T.K. and S.Y.; formal analysis, T.K.; investigation, T.K.; resources, T.K.; data curation, T.K.; writing—original draft preparation, T.K.; writing—review and editing, S.Y.; visualization, T.K.; supervision, S.Y.; project administration, S.Y.; and funding acquisition, S.Y. All authors have read and agreed to the published version of the manuscript.

Funding: This research was supported by Wonkwang University in 2025.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available upon reasonable request from the corresponding author.

Acknowledgments: The authors thank the anonymous reviewers and editors for their insightful comments and suggestions.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- McMahan, H.B.; Moore, E.; Ramage, D.; Hampson, S. Communication-efficient learning of deep networks from decentralized data. *arXiv* **2016**, arXiv:1602.05629.
- Kairouz, P.; McMahan, H.B.; Avent, B.; Bellet, A.; Bennis, M.; Bhagoji, A.N.; Bonawitz, K.; Charles, Z.; Cormode, G.; Cummings, R.; et al. Advances and open problems in federated learning. *Found. Trends Mach. Learn.* **2021**, *14*, 1–210. [\[CrossRef\]](#)
- Alazab, M.; Khan, S.; Krishnan, S.S.R.; Pham, Q.-V.; Reddy, M.P.K.; Gadekallu, T.R. Federated Learning for Cybersecurity: Concepts, Challenges and Future Directions. *IEEE Trans. Ind. Inform.* **2021**, *18*, 3501–3509. [\[CrossRef\]](#)
- Hard, A.; Rao, K.; Mathews, R.; Ramaswamy, S.; Beaufays, F.; Augenstein, S.; Eichner, H.; Kiddon, C.; Ramage, D. Federated learning for mobile keyboard prediction. *arXiv* **2019**, arXiv:1811.03604.
- Fan, L.; Jin, H. A Practical Framework for Privacy-Preserving Data Analytics. In Proceedings of the 24th International Conference on World Wide Web, Florence, Italy, 18–22 May 2015; pp. 311–321.
- Hinton, G.; Vinyals, O.; Dean, J. Distilling the knowledge in a neural network. *arXiv* **2015**, arXiv:1503.02531.
- Zhao, Y.; Li, M.; Lai, L.; Suda, N.; Civin, D.; Chandra, V. Federated Learning with Non-IID Data. *arXiv* **2018**, arXiv:1806.00582. [\[CrossRef\]](#)
- Reddi, S.J.; Charles, Z.; Zaheer, M.; Garrett, Z.; Rush, K.; Konečný, J.; Kumar, S.; McMahan, H.B. Adaptive Federated Optimization. *arXiv* **2020**, arXiv:2003.00295.
- Li, T.; Hu, S.; Beirami, A.; Smith, V. Ditto: Fair and Robust Federated Learning Through Personalization. *arXiv* **2020**, arXiv:2012.04221.
- Neyigapula, B.S. Federated Learning for Collaborative Network Security in Decentralized Environments. *J. Sens. Netw. Data Commun.* **2023**, *3*, 81–92.
- Hanzely, F.; Richtárik, P. Federated learning of a mixture of global and local models. *arXiv* **2021**, arXiv:2002.05516.
- Tharaphe, T.T.; Yoshiaki, S.; Masakatu, M. Personalized federated learning-based intrusion detection system: Poisoning attack and defense. *Future Gener. Comput. Syst.* **2024**, *149*, 44–58.
- Chakraborty, S.; Sharma, N.; Mondal, T.; Khatri, S.K.; Biswas, B. Machine Learning for Automated Industrial IoT Attack Detection: An Efficiency-Complexity Trade-off. *ACM Trans. Manag. Inf. Syst.* **2021**, *12*, 1–26. [\[CrossRef\]](#)
- Hsu, T.-M.H.; Qi, H.; Brown, M. Measuring the effects of non-identical data distribution for federated visual classification. *arXiv* **2019**, arXiv:1909.06335.
- Khodak, M.; Balcan, M.-F.; Talwalkar, A. Adaptive gradient-based meta-learning methods. *arXiv* **2019**, arXiv:1906.02717.
- Arivazhagan, M.G.; Aggarwal, V.; Singh, A.K.; Choudhary, S. Federated learning with personalization layers. *arXiv* **2019**, arXiv:1912.00818.
- Dinh, C.T.; Tran, N.H.; Nguyen, T.D. Personalized Federated Learning with Moreau Envelopes. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020), Virtual Conference, 6–12 December 2020; pp. 21394–21405.
- Shamsian, A.; Navon, A.; Fetaya, E.; Chechik, G. Personalized Federated Learning using Hypernetworks. *arXiv* **2021**, arXiv:2103.04628.
- Shibly, K.H.; Roy, S.K.; Hossain, M.A.; Bhuiyan, M.Z.A.; Hossain, M.S. Personalized Federated Learning for Automotive Intrusion Detection Systems. In Proceedings of the 2022 IEEE Future Networks World Forum, Montreal, QC, Canada, 12–14 October 2022; pp. 544–549.
- He, C.; Annaram, M.; Avestimehr, S. Group Knowledge Transfer: Federated Learning of Large CNNs at the Edge. In Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS 2020), Virtual Conference, 6–12 December 2020; pp. 14068–14080.
- Lin, T.; Stich, S.U.; Patel, K.K.; Jaggi, M. Don't Use Large Mini-Batches, Use Local SGD. In Proceedings of the 8th International Conference on Learning Representations (ICLR 2020), Addis Ababa, Ethiopia, 26–30 April 2020.
- Sengupta, S.; Chowdhary, A.; Sabur, A.; Alshamrani, A.; Huang, D.; Kang, S. Privacy-Preserving Cyber-Security Information Exchange Mechanism. In Proceedings of the Performance Evaluation of Computer and Telecommunication Systems, Montreal, QC, Canada, 24–27 July 2016.
- Alkhamisi, A.; Katib, I.; Buhari, S.M. Federated Learning-Based Security Attack Detection for Multi-Controller Software-Defined Networks. *Algorithms* **2024**, *17*, 290. [\[CrossRef\]](#)
- Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*; Technical Report; University of Toronto: Toronto, ON, Canada, 2009. Available online: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf> (accessed on 29 April 2025).
- Fallah, A.; Mokhtari, A.; Ozdaglar, A. Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach. In Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS 2020), Virtual Conference, 6–12 December 2020; pp. 3557–3568.
- Li, T.; Sanjabi, M.; Beirami, A.; Smith, V. Fair Resource Allocation in Federated Learning. In Proceedings of the 8th International Conference on Learning Representations (ICLR 2020), Addis Ababa, Ethiopia, 26–30 April 2020.
- He, H.; Ma, Y. *Imbalanced Learning: Foundations, Algorithms, and Applications*; Wiley-IEEE Press: Hoboken, NJ, USA, 2013.

28. Buda, M.; Maki, A.; Mazurowski, M.A. A systematic study of the class imbalance problem in convolutional neural networks. *Neural Netw.* **2018**, *106*, 249–259. [[CrossRef](#)]
29. Chen, C.; Liu, X.; Qiu, T.; Sangaiah, A.K. Dynamic oversampling strategy for imbalanced big data in intrusion detection. *IEEE Access* **2021**, *9*, 99362–99375.
30. Collins, L.; Hassani, H.; Mokhtari, A.; Shakkottai, S. Exploiting shared representations for personalized federated learning. *arXiv* **2023**, arXiv:2102.07078.
31. Wu, D.; Xia, S.-T.; Wang, Y. Adversarial Weight Perturbation Helps Robust Generalization. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 11964–11976.
32. Smith, V.; Chiang, C.-K.; Sanjabi, M.; Talwalkar, A.S. Federated Multi-Task Learning. In Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS 2017), Long Beach, CA, USA, 4–9 December 2017; pp. 4424–4434.
33. Caldas, S.; Wu, P.; Li, T.; Konečný, J.; McMahan, H.B.; Smith, V.; Talwalkar, A. LEAF: A benchmark for federated settings. *arXiv* **2019**, arXiv:1812.01097.
34. Moustafa, N.; Slay, J. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set). In Proceedings of the Military Communications and Information Systems Conference (MilCIS), Canberra, ACT, Australia, 10–12 November 2015; pp. 1–6. Available online: <https://research.unsw.edu.au/projects/unswnb15-dataset> (accessed on 19 March 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.